

**CISS362: Introduction to Automata Theory, Languages, and
Computation
Assignment a09**

Name: _____

OBJECTIVES

- Design context-free grammars.
- Design PDA.

For questions asking you show an operation on context-free languages language is closed, you need not prove your construction is correct. (But you are welcome to do so. Most of such proofs, if not immediate, is by induction.) If the question does not say so, you can choose to give a PDA or a context-free grammar. For a PDA, draw the PDA state diagram and give the formal definition.

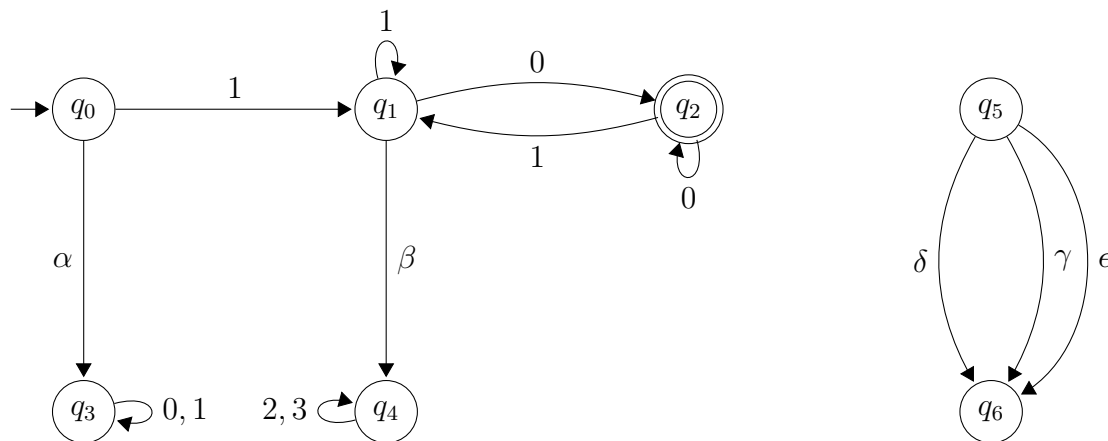
Study the solutions to the following questions in the textbook: 2.3, 2.4 (a) and (d), 2.6(a) and (c), 2.7, 2.8.

Here are the questions you should work. Some questions have solutions (either from the book or I have written up the solution).

- Sipser 2.1. Q1. Solution to 2.1(b) is provided. Study it carefully.
- Sipser 2.2. Q2. You may assume that $\{a^n b^n c^n \mid n \geq 0\}$ is not a context-free language.
- Sipser 2.3. Solution is provided in the textbook. Study it carefully.
- Sipser 2.4. Q3. Solutions to 2.4(a) and 2.4(d) are provided in the textbook. Study it carefully.
- Sipser 2.5. Q4.
- Sipser 2.6. Q5. Solutions to 2.6(a) and 2.6(c) are provided in the textbook. Study it carefully.
- Sipser 2.7. Solution to 2.7 is provided in the textbook. Study it carefully.
- Sipser 2.8. Solution to 2.8 is provided in the textbook. Study it carefully.
- Sipser 2.9. Q6.
- Sipser 2.10. Q7.
- Sipser 2.13. Q8. Solution to 2.13(a) is provided.

HOW TO DRAW A STATE DIAGRAM

Here's an example showing you how to draw the elements of a state diagram. Also, look at the solution to 1.3 below.



For more information on drawing state diagrams go to my tutorials at <http://yliow.github.io> and look for `latex-automata.pdf`: Let me know if you have any questions about drawing state diagram.

HOW TO WRITE A CONTEXT-FREE GRAMMAR

Here's a context-free grammar, G , for our $\{a^n b^n \mid n \geq 0\}$:

$$G : \begin{cases} S \rightarrow \epsilon \\ S \rightarrow aSb \end{cases}$$

Usually we combine the productions for the same variable like this:

$$G : S \rightarrow \epsilon \mid aSb$$

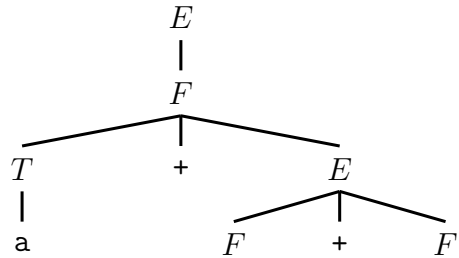
Here's an example of a longer grammar:

$$G_1 : \begin{cases} S \rightarrow \epsilon \mid TU \\ T \rightarrow a \mid aT \\ U \rightarrow \epsilon \mid bUcc \end{cases}$$

And remember to list the starting variable first.

HOW TO DRAW A PARSE TREE

Here's an example on how to draw a parse tree:



Here's the python code in `main.tex` that draws the above:

```

\begin{python}
from latextool_basic import *
p = Plot()

d = positions(r"""
  A
  B
C D E
F G H I
""", xscale=0.7)
edges = {'A':['B'],
        'B':['C', 'D', 'E'],
        'C':['F'],
        'E':['G', 'H', 'I'],
        }
labels = {'A':r'$E$',
        'B':r'$F$',
        'C':r'$T$',
        'D':r'\texttt{+}',
        'E':r'$E$',
        'F':r'\texttt{a}',
        'G':r'$F$',
        'H':r'\texttt{+}',
        'I':r'$F$',
        }

def draw(p, d, edges):
    rects = {}
    for k,(x,y) in d.items():
        rects[k] = Rect(x0=x, y0=y-0.3, x1=x, y1=y+0.3, label=labels[k],
                        name=k, linecolor='white')
    p += rects[k]
    for k,v in edges.items():
        for _ in v:
            p += Line(points=[rects[k].bottom(), rects[_].top()])

draw(p, d, edges)
print(p)
\end{python}

```

To draw a parse tree, copy-and-paste the above from `main.tex` and modify it. The python code will generate the latex code for you. Here's a quick explanation of the python code ...

In the python code, the

```

    A
    B
  C  D  E
F   G H I

```

are places where you want to place nodes of the parse tree. You use characters to mark these places. In the python code, the

```

edges = {'A': ['B'],
        'B': ['C', 'D', 'E'],
        'C': ['F'],
        'E': ['G', 'H', 'I'],
        }

```

describe the edges. For instance B has children C, D, E. In the python code, the

```

labels = {'A': r'$E$',
        'B': r'$F$',
        'C': r'$T$',
        'D': r'\texttt{+}',
        'E': r'$E$',
        'F': r'\texttt{a}',
        'G': r'$F$',
        'H': r'\texttt{+}',
        'I': r'$F$',
        }

```

describes what to print for the nodes using latex fragments. For instance for A, you'll see *E*. The `\texttt{a}` means to use typewriter font for a.

You need not change the python code after labels.

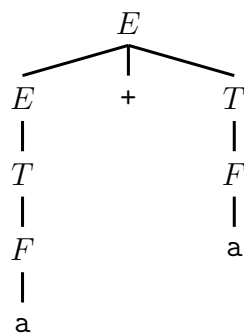
Q1. Sipser 2.1.

SOLUTION.

(a)

(b)

$$\begin{aligned}
 E &\Rightarrow E+T \\
 &\Rightarrow T+T \\
 &\Rightarrow F+T \\
 &\Rightarrow a+T \\
 &\Rightarrow a+F \\
 &\Rightarrow a+a
 \end{aligned}$$



(c)

(d)

Q2. Sipser 2.2.

SOLUTION.

Q3. Sipser 2.4.(b), (c), (e), (f)

SOLUTION.

Q4. Sipser 2.5. For each questions, draw PDA state diagram and informally describe how it words. You do *not* need to give the formal definition.

SOLUTION.

Q5. Sipser 2.6 (b) and (d).

SOLUTION.

Q6. Sipser 2.9.

SOLUTION.

Q7. Sipser 2.10. (Hint: Look at the word “or”.)

SOLUTION.

Q8. Sipser 2.13.

SOLUTION.

(a) T derives strings containing any number of 0s (including none) and exactly one $\#$. Therefore TT derives strings with any number of 0s and exactly two $\#$ s. U derives strings of the form $0^n\#0^{2n}$. Hence $L(G)$ contains strings with any number of 0s and exactly two $\#$ or strings of the form $0^n\#0^{2n}$, i.e.,

$$L(G) = L(0^*\#0^*\#0^*) \cup \{0^n\#0^{2n} \mid n \geq 0\}$$